

COM S 227: Object Oriented Programming

Homework 3, 100 points

Due Mon Jul 1st (Summer 2024)

General Information

- NO LATE SUBMISSIONS ALLOWED
- PLEASE FOLLOW INSTRUCTIONS CAREFULLY!
- This assignment is to be done ON YOUR OWN. See the Academic Dishonesty policy in the syllabus for details. If you need help, see your instructor or one of the TAs. Lots of help is also available through the Piazza discussions.

Overview

This is a relatively short assignment to have some fun and give you some practice with recursion. Your task is to implement the recursive method `generateTarget` in the class `TwentyFour`.

This is loosely based on a well-known game called the “twenty-four game”. The idea is that you are given a list of numbers and a target value (in the traditional form of the game, the target value is always 24). For example, suppose your list of numbers is 2, 3, 4, 5, with a target value of 21.

Your GOAL is to combine your numbers using addition, subtraction, multiplication, division and exponent to obtain the target (implicitly you are also using parentheses since you can group the operations). For example, in the above example - some possible solutions would be

- $((5 - 2) * (3 + 4)),$
- $((3 - 2) + (4 * 5)),$
- $(3 * (2 + 5)),$
- $((2 ^ 4) + 5)$

and so on. We are using $^$ to indicate exponent i.e. power operation. All of them would result in the answer 21. Note that you can use each number in the given list ONLY once. You can assume that each number in the list is unique. You can use the operators any number of times (including zero times). Note that in our version of the rules, you are not required to use all the numbers, and division is only allowed when there is no remainder. NOTE: This is different from the traditional version.

To achieve the goal, you will create code for the `generateTarget` method described below.

The `generateTarget` method is to create a list containing all possible combinations, represented as strings. It is ok for the list to contain duplicates and the results do not need to be in any particular order. Please note, for example, that we regard “ $(3 * (2 + 5))$ ” and “ $(3 * (5 + 2))$ ” as *different* solutions (this is to simplify the coding).

Solution Idea

The solution idea is to decompose the problem into smaller sub-problems so that recursion can be used.

There are two possible ways to reduce the problem to a smaller sub-problem.

- a) Strategy a: Choose two numbers a and b in the list, remove them from the list, and replace them with a single number which is a result of either of $a + b$, $b + a$, $a * b$, $b * a$, $a - b$, $b - a$, a / b , b / a , a^b , b^a . This reduces the list size by 1. (Remember, in java, the operation a^b is written using `Math.pow(a, b)`).
- b) Strategy b: Choose a single number in the list and remove it. This allows us to find solutions in which not all the numbers are used; again, the list size is reduced by 1.

The base case is easy: if you have a list of size 1, you can just check whether that value is equal to the target.

Using Strategy (a)

The idea is illustrated in the example using the list of numbers 2, 3, 4, 5 and the target value 11. We start by choosing a pair of values from the list. In this example we are examining the path in which we chose 2 and 5.

We can replace the 2 and the 5 with

- 7 ($2 + 5$ or $5 + 2$),
- 10 ($2 * 5$ or $5 * 2$),
- -3 ($2 - 5$),
- 3 ($5 - 2$),
- 32 (2^5),
- 25 (5^2).

Both $5/2$ and $2 / 5$ are disallowed because they have a non-zero remainder.

This leads to sublists (7, 3, 4), (10, 3, 4), (-3, 3, 4), (3, 3, 4), (32, 3, 4), and (25, 3, 4) each of which are smaller in size (i.e. now the size is 3). The same strategy can now be recursively applied to each of the three-element lists.

Using Strategy (b)

We can form subproblems just by removing an element and obtaining a shorter list, to obtain a solution that doesn't use all the numbers such as (3, 4, 5), (2, 4, 5), (2, 3, 5), and (2, 3, 4). These are smaller in size.

Pseudocode

```
if the list has length 1
    if the value matches the target
        add it to the list of results.
otherwise
    // strategy (B)
    for each number x in the list
        create a copy of the list that does not include x
        find solutions using that list.
    // strategy (A)
    for each pair of numbers x, y in the list
        for each allowable arithmetic combination z of x and y
            create a copy of the list without x and y, but with z added.
            find solutions using that list.
```

IMPORTANT When we say, “each allowable arithmetic combination z of x and y”, we always include $x + y$, $y + x$, $x * y$, $y * x$, $x - y$, $y - x$, $x ^ y$ and $y ^ x$. For division, x / y or y / x is only included when there is no remainder and no division by zero. (Having $x + y$ and $y + x$ is redundant, but it simplifies the code and the testing to always include them both.)

There is an additional question. When we reach a base case and check that the target value has been found, how do we know what the operations were that led to it? **Somehow, we should store not only the numbers in the list, but also the arithmetic expressions that were used to obtain them.** For example, in the first step when we compute $2 * 5 = 10$, we need to save not only the number 10, but also something like the string “(2 * 5)”.

A `TwentyFour` object just contains a number and a string. You can construct a `TwentyFour` with a single integer or by combining two existing `TwentyFour` objects with one of the operators ‘+’, ‘*’, ‘-’, ‘/’ or ‘^’. Doing so automatically computes the result and stores a new string representation of the old values. Parentheses are always added. You obtain the integer value using the method `intValue()` and the string form using the method `toString()`. Here are some examples:

```
TwentyFour v1 = new TwentyFour(2);
TwentyFour v2 = new TwentyFour(3);
System.out.println(v1.toString()); // prints "2"
System.out.println(v2.toString()); // prints "3"
TwentyFour v3 = new TwentyFour(v1, v2, '+');
System.out.println(v3.toString()); // prints "(2 + 3)"
System.out.println(v3.intValue()); // 5
TwentyFour v4 = new TwentyFour(v2, v3, '*');
System.out.println(v4.toString()); // prints "(3 * (2 + 3))"
```

Instead of using an `ArrayList<Integer>`, we will use `ArrayList<TwentyFour>` to represent a list of values. If you wanted to run the example in Figure 1, you could use a main method with statements such as the following:

```
ArrayList<TwentyFour> expressions = new ArrayList<TwentyFour>();
for (int x : values)
    expressions.add(new TwentyFour(x));
ArrayList<String> results = new ArrayList<String>(); // empty list
TwentyFour.generateTarget(expressions, target, results);
```

Note that this implies that `generateTarget` is a static method in `TwentyFour` with the following signature:

```
public void generateTarget(ArrayList<TwentyFour> expressions, int target, ArrayList<String> results)
```

To view the solutions after they have been generated, it is probably best to remove duplicates and sort first:

```
ArrayList<String> uniqueResults = new ArrayList<String>();
for (String r : results)
    if (!uniqueResults.contains(r))
        uniqueResults.add(r)

Collections.sort(uniqueResults);
for (String r : uniqueResults)
    System.out.println(r);
```

For example, the complete list of solutions for the list 2, 3, 3, 5 with target 11 (after removing duplicates and sorting as strings) is:

```
((3 ^ 2) + 5) - 3
((3 ^ 2) - 3) + 5
((3 ^ 3) - 5) / 2
(2 * 3) + 5
(2 * 5) + (3 / 3)
(2 ^ 3) + 3
(3 * 2) + 5
(3 * 3) + 2
(3 + 3) + 5
(3 + 5) + 3
(3 / 3) + (2 * 5)
(3 / 3) + (5 * 2)
(3 ^ (5 - 3)) + 2
(3 ^ 2) + (5 - 3)
(3 ^ 2) - (3 - 5)
(5 * 2) + (3 / 3)
(5 + (3 ^ 2)) - 3
(5 + 3) + 3
(5 - 3) + (3 ^ 2)
2 + (3 * 3)
2 + (3 ^ (5 - 3))
3 + (2 ^ 3)
3 + (3 + 5)
3 + (5 + 3)
5 + ((3 ^ 2) - 3)
5 + (2 * 3)
5 + (3 * 2)
5 + (3 + 3)
5 - (3 - (3 ^ 2))
Size: 29
```

The expression “ $(2 + 3)$ ” is considered to be different from the expression “ $(3 + 2)$ ”. Your implementation should find all solutions and therefore needs to try both. Note: your solution can contain duplicates and the order doesn’t matter. (Our unit tests will remove duplicates and sort before checking against a correct solution.)

Documentation and style

The grading is automated and, in most cases, we will not be reading your code - although we will send it to a automated plagiarism checker. Do not use any static variables in your class.

If you have questions, ask on Piazza under HW3. Keep in mind the Academic Dishonesty Policies. Do not post any source code (except privately on your own private channel).

Getting started

1. Recursion is one of the harder topics in this class. **You should start asap asap asap.** First do all the practice/challenge exercises. Also, go over your recent lab to see how to use the debugger. **Work out some small examples on paper by hand.** Example (2, 3, 5) and target 22
2. You should start with a simpler version of the problem just to make sure you have the recursion working correctly. For example, suppose that all numbers must be used, and that only addition is allowed. See if you can do an example such as the set 1, 2 with target 3 (2 solutions). Then you might try 1, 2, 3 with target 3 (17 solutions).

What to turn in

Please submit, on GradeScope, the file `TwentyFour.java` (only). It should be in the package `hw3`.